

Report conversation

If I have a variable in Go that's a pointer, do I need a lock?
For example, suppose we have something like the following:

```
var X *MyStruct{}

go func(){
    for {
        X = NewMyStruct()
    }
}()

go func(){
    tempX = X
    // Print something based on tempX
}()
```

Is a lock required here?

In the example you've provided, where one goroutine continuously updates a pointer `x` to point to a new instance of `MyStruct` and another goroutine reads from this pointer, you most likely need to use synchronization mechanisms such as a lock to avoid potential data races and ensure memory visibility between goroutines.

In Go, the memory model does not guarantee that changes made by one goroutine to shared variables (like your pointer `x`) are immediately visible to other goroutines without proper synchronization. This means the second goroutine might not see the updates to `x` in a timely manner, or worse, it might see a partially updated state if the write and read happen simultaneously, leading to unpredictable behavior.

Using a lock, such as `sync.Mutex`, ensures that only one goroutine can access the variable at a time, thus preventing data races. Here's how you might modify your example to use a `sync.Mutex` for safe access to `x`:

```
import (  
    "sync"  
)  
  
var (  
    X      *MyStruct  
    lock sync.Mutex  
)  
  
// Goroutine for updating X  
go func() {  
    for {  
        lock.Lock()  
        X = NewMyStruct()  
        lock.Unlock()  
    }  
}()  
  
// Goroutine for reading X  
go func() {  
    for {  
        lock.Lock()  
        tempX := X  
        lock.Unlock()  
        // Print something based on tempX  
    }  
}()
```

This ensures that when one goroutine is updating or reading `x`, no other goroutine can access `x` at the same time, preventing race conditions and ensuring that the read goroutine always gets a valid state of `x`.

However, using locks comes with its own set of challenges, such as potential deadlocks, reduced performance due to blocking, and the complexity of managing locks correctly. Depending on your specific use case, other concurrency mechanisms provided by Go, such as channels or atomic operations from the `sync/atomic` package, might be more appropriate and could offer better performance or simplicity. Atomic operations, for

example, can be used for simple read-modify-write operations on primitive types without the overhead of locking, but they require careful use to ensure correctness.

Is it possible for `tempX` to point to something other than a prior instance of `X`, or can it be a completely random value if the pointer-copy happens at precisely the wrong time?

In Go, if a pointer operation (like copying the pointer `x` to `tempX`) happens without proper synchronization and concurrently with updates to that pointer, the behavior can indeed be unpredictable due to the data race. However, under typical circumstances and assuming a well-behaved Go runtime and hardware, `tempX` will point to either the value of `x` before or after the update, not to a completely random or uninitialized memory location. The Go memory model and the hardware it runs on ensure basic atomicity for pointer assignments, meaning a pointer value is either fully updated or not updated at all from the perspective of a single CPU core executing these instructions.

In the context of your example:

go

 Copy code

```
go func() {  
    for {  
        X = NewMyStruct() // Update pointer X to a new instance  
    }  
}()  
  
go func() {  
    tempX = X // Copy X to tempX  
    // Print something based on tempX  
}()
```

There are a few scenarios regarding `tempX`:

- 1. Before the Update:** `tempX` captures the value of `x` before it is updated by the first goroutine.

2. **After the Update:** `tempX` captures the value of `x` after the update.
3. **Data Race Undefined Behavior:** In theory, due to the data race, the behavior is undefined within the Go memory model, which might suggest a range of unpredictable outcomes. However, in practice, because of the way modern computers work, `tempX` is highly unlikely to point to a completely arbitrary or invalid memory location due to the atomic nature of pointer assignments on most platforms. A data race might cause your program to see an old or new value, but not a random address.

However, "undefined behavior" in the context of data races means that the program's behavior is not predictable or guaranteed by the language's memory model. It doesn't necessarily mean that the program will do something entirely nonsensical like assigning a random memory address to `tempX`, but it does mean the program's correctness cannot be assured, and it might behave differently across runs, compiler versions, or hardware.

The key takeaway is that while you might not get a "completely random value" in the sense of pointing to arbitrary memory, accessing shared memory without proper synchronization in concurrent programming can lead to subtle, hard-to-debug errors. It's always best practice to use synchronization mechanisms like mutexes, channels, or atomic operations to ensure memory visibility and avoid data races, ensuring that your program behaves correctly and predictably.

What does this doc say about copying a pointer?

Go
Skip to Main Content
Why Go
arrow_drop_down
Learn
Docs
arrow_drop_down
Packages
Community
arrow_drop_down
Go.
Why Go
navigate_next

navigate_beforeWhy Go
Case Studies
Use Cases
Security
Learn
Docsnavigate_next
navigate_beforeDocs
Effective Go
Go User Manual
Standard library
Release Notes
Packages
Communitynavigate_next
navigate_beforeCommunity
Recorded Talks
Meetups open_in_new
Conferences open_in_new
Go blog
Go project
Get connected

The Go Memory Model
Version of June 6, 2022
Table of Contents
Introduction
Advice
Informal Overview
Memory Model
Implementation Restrictions for Programs Containing
Data Races
Synchronization
Initialization
Goroutine creation
Goroutine destruction
Channel communication

Locks

Once

Atomic Values

Finalizers

Additional Mechanisms

Incorrect synchronization

Incorrect compilation

Conclusion

Introduction

The Go memory model specifies the conditions under which reads of a variable in one goroutine can be guaranteed to observe values produced by writes to the same variable in a different goroutine.

Advice

Programs that modify data being simultaneously accessed by multiple goroutines must serialize such access.

To serialize access, protect the data with channel operations or other synchronization primitives such as those in the sync and sync/atomic packages.

If you must read the rest of this document to understand the behavior of your program, you are being too clever.

Don't be clever.

Informal Overview

Go approaches its memory model in much the same way as the rest of the language, aiming to keep the semantics simple, understandable, and useful. This section gives a general overview of the approach and should suffice for most programmers. The memory model is specified more formally in the next section.

A data race is defined as a write to a memory location happening concurrently with another read or write to that same location, unless all the accesses involved are atomic data accesses as provided by the sync/atomic package. As noted already, programmers are strongly encouraged to

use appropriate synchronization to avoid data races. In the absence of data races, Go programs behave as if all the goroutines were multiplexed onto a single processor. This property is sometimes referred to as DRF-SC: data-race-free programs execute in a sequentially consistent manner.

While programmers should write Go programs without data races, there are limitations to what a Go implementation can do in response to a data race. An implementation may always react to a data race by reporting the race and terminating the program. Otherwise, each read of a single-word-sized or sub-word-sized memory location must observe a value actually written to that location (perhaps by a concurrent executing goroutine) and not yet overwritten. These implementation constraints make Go more like Java or JavaScript, in that most races have a limited number of outcomes, and less like C and C++, where the meaning of any program with a race is entirely undefined, and the compiler may do anything at all. Go's approach aims to make errant programs more reliable and easier to debug, while still insisting that races are errors and that tools can diagnose and report them.

Memory Model

The following formal definition of Go's memory model closely follows the approach presented by Hans-J. Boehm and Sarita V. Adve in "Foundations of the C++ Concurrency Memory Model", published in PLDI 2008. The definition of data-race-free programs and the guarantee of sequential consistency for race-free programs are equivalent to the ones in that work.

The memory model describes the requirements on program executions, which are made up of goroutine executions, which in turn are made up of memory operations.

A memory operation is modeled by four details:

its kind, indicating whether it is an ordinary data read, an ordinary data write, or a synchronizing operation such as an atomic data access, a mutex operation, or a channel operation,

its location in the program,

the memory location or variable being accessed, and

the values read or written by the operation.

Some memory operations are read-like, including read, atomic read, mutex lock, and channel receive. Other

memory operations are write-like, including write, atomic write, mutex unlock, channel send, and channel close.

Some, such as atomic compare-and-swap, are both read-like and write-like.

A goroutine execution is modeled as a set of memory operations executed by a single goroutine.

Requirement 1: The memory operations in each goroutine must correspond to a correct sequential execution of that goroutine, given the values read from and written to memory. That execution must be consistent with the sequenced before relation, defined as the partial order requirements set out by the Go language specification for Go's control flow constructs as well as the order of evaluation for expressions.

A Go program execution is modeled as a set of goroutine executions, together with a mapping W that specifies the write-like operation that each read-like operation reads from. (Multiple executions of the same program can have different program executions.)

Requirement 2: For a given program execution, the mapping W , when limited to synchronizing operations, must be explainable by some implicit total order of the synchronizing operations that is consistent with sequencing and the values read and written by those operations.

The synchronized before relation is a partial order on synchronizing memory operations, derived from W . If a synchronizing read-like memory operation r observes a synchronizing write-like memory operation w (that is, if $W(r) = w$), then w is synchronized before r . Informally, the synchronized before relation is a subset of the implied total order mentioned in the previous paragraph, limited to the information that W directly observes.

The happens before relation is defined as the transitive closure of the union of the sequenced before and synchronized before relations.

Requirement 3: For an ordinary (non-synchronizing) data read r on a memory location x , $W(r)$ must be a write w that is visible to r , where visible means that both of the following hold:

- w happens before r .

- w does not happen before any other write w' (to x) that happens before r .

A read-write data race on memory location x consists of a read-like memory operation r on x and a write-like memory operation w on x , at least one of which is non-synchronizing, which are unordered by happens before (that is, neither r happens before w nor w happens before r).

A write-write data race on memory location x consists of two write-like memory operations w and w' on x , at least one of which is non-synchronizing, which are unordered by happens before.

Note that if there are no read-write or write-write data races on memory location x , then any read r on x has only one possible $W(r)$: the single w that immediately precedes it in the happens before order.

More generally, it can be shown that any Go program that is data-race-free, meaning it has no program executions

with read-write or write-write data races, can only have outcomes explained by some sequentially consistent interleaving of the goroutine executions. (The proof is the same as Section 7 of Boehm and Adve's paper cited above.) This property is called DRF-SC.

The intent of the formal definition is to match the DRF-SC guarantee provided to race-free programs by other languages, including C, C++, Java, JavaScript, Rust, and Swift.

Certain Go language operations such as goroutine creation and memory allocation act as synchronization operations. The effect of these operations on the synchronized-before partial order is documented in the “Synchronization” section below. Individual packages are responsible for providing similar documentation for their own operations.

Implementation Restrictions for Programs Containing Data Races

The preceding section gave a formal definition of data-race-free program execution. This section informally describes the semantics that implementations must provide for programs that do contain races.

First, any implementation can, upon detecting a data race, report the race and halt execution of the program. Implementations using ThreadSanitizer (accessed with “go build -race”) do exactly this.

Otherwise, a read r of a memory location x that is not larger than a machine word must observe some write w such that r does not happen before w and there is no write w' such that w happens before w' and w' happens before r . That is, each read must observe a value written by a preceding or concurrent write.

Additionally, observation of acausal and “out of thin air” writes is disallowed.

Reads of memory locations larger than a single machine word are encouraged but not required to meet the same semantics as word-sized memory locations, observing a single allowed write *w*. For performance reasons, implementations may instead treat larger operations as a set of individual machine-word-sized operations in an unspecified order. This means that races on multiword data structures can lead to inconsistent values not corresponding to a single write. When the values depend on the consistency of internal (pointer, length) or (pointer, type) pairs, as can be the case for interface values, maps, slices, and strings in most Go implementations, such races can in turn lead to arbitrary memory corruption.

Examples of incorrect synchronization are given in the “Incorrect synchronization” section below.

Examples of the limitations on implementations are given in the “Incorrect compilation” section below.

Synchronization

Initialization

Program initialization runs in a single goroutine, but that goroutine may create other goroutines, which run concurrently.

If a package *p* imports package *q*, the completion of *q*'s init functions happens before the start of any of *p*'s.

The completion of all init functions is synchronized before the start of the function `main.main`.

Goroutine creation

The `go` statement that starts a new goroutine is synchronized before the start of the goroutine's execution.

For example, in this program:

```
var a string
```

```
func f() {  
    print(a)  
}
```

```
func hello() {  
    a = "hello, world"  
    go f()  
}
```

calling hello will print "hello, world" at some point in the future (perhaps after hello has returned).

Goroutine destruction

The exit of a goroutine is not guaranteed to be synchronized before any event in the program. For example, in this program:

```
var a string
```

```
func hello() {  
    go func() { a = "hello" }()  
    print(a)  
}
```

the assignment to a is not followed by any synchronization event, so it is not guaranteed to be observed by any other goroutine. In fact, an aggressive compiler might delete the entire go statement.

If the effects of a goroutine must be observed by another goroutine, use a synchronization mechanism such as a lock or channel communication to establish a relative ordering.

Channel communication

Channel communication is the main method of synchronization between goroutines. Each send on a particular channel is matched to a corresponding receive from that channel, usually in a different goroutine.

A send on a channel is synchronized before the completion of the corresponding receive from that channel.

This program:

```
var c = make(chan int, 10)
var a string
```

```
func f() {
    a = "hello, world"
    c <- 0
}
```

```
func main() {
    go f()
    <-c
    print(a)
}
```

is guaranteed to print "hello, world". The write to `a` is sequenced before the send on `c`, which is synchronized before the corresponding receive on `c` completes, which is sequenced before the print.

The closing of a channel is synchronized before a receive that returns a zero value because the channel is closed.

In the previous example, replacing `c <- 0` with `close(c)` yields a program with the same guaranteed behavior.

A receive from an unbuffered channel is synchronized before the completion of the corresponding send on that channel.

This program (as above, but with the send and receive statements swapped and using an unbuffered channel):

```
var c = make(chan int)
var a string
```

```
func f() {  
    a = "hello, world"  
    <-c  
}
```

```
func main() {  
    go f()  
    c <- 0  
    print(a)  
}
```

is also guaranteed to print "hello, world". The write to a is sequenced before the receive on c, which is synchronized before the corresponding send on c completes, which is sequenced before the print.

If the channel were buffered (e.g., `c = make(chan int, 1)`) then the program would not be guaranteed to print "hello, world". (It might print the empty string, crash, or do something else.)

The k th receive on a channel with capacity C is synchronized before the completion of the $k+C$ th send from that channel completes.

This rule generalizes the previous rule to buffered channels. It allows a counting semaphore to be modeled by a buffered channel: the number of items in the channel corresponds to the number of active uses, the capacity of the channel corresponds to the maximum number of simultaneous uses, sending an item acquires the semaphore, and receiving an item releases the semaphore. This is a common idiom for limiting concurrency.

This program starts a goroutine for every entry in the work list, but the goroutines coordinate using the limit channel to ensure that at most three are running work functions at a time.

```
var limit = make(chan int, 3)
```

```
func main() {
    for _, w := range work {
        go func(w func()) {
            limit <- 1
            w()
            <-limit
        }(w)
    }
    select{}
}
```

Locks

The sync package implements two lock data types, sync.Mutex and sync.RWMutex.

For any sync.Mutex or sync.RWMutex variable l and $n < m$, call n of l.Unlock() is synchronized before call m of l.Lock() returns.

This program:

```
var l sync.Mutex
var a string

func f() {
    a = "hello, world"
    l.Unlock()
}
```

```
func main() {
    l.Lock()
    go f()
    l.Lock()
    print(a)
}
```

is guaranteed to print "hello, world". The first call to l.Unlock() (in f) is synchronized before the second call to l.Lock() (in main) returns, which is sequenced before the print.

For any call to `l.RLock` on a `sync.RWMutex` variable `l`, there is an `n` such that the `n`th call to `l.Unlock` is synchronized before the return from `l.RLock`, and the matching call to `l.RUnlock` is synchronized before the return from call `n+1` to `l.Lock`.

A successful call to `l.TryLock` (or `l.TryRLock`) is equivalent to a call to `l.Lock` (or `l.RLock`). An unsuccessful call has no synchronizing effect at all. As far as the memory model is concerned, `l.TryLock` (or `l.TryRLock`) may be considered to be able to return false even when the mutex `l` is unlocked.

Once

The `sync` package provides a safe mechanism for initialization in the presence of multiple goroutines through the use of the `Once` type. Multiple threads can execute `once.Do(f)` for a particular `f`, but only one will run `f()`, and the other calls block until `f()` has returned.

The completion of a single call of `f()` from `once.Do(f)` is synchronized before the return of any call of `once.Do(f)`.

In this program:

```
var a string
var once sync.Once

func setup() {
    a = "hello, world"
}

func doprint() {
    once.Do(setup)
    print(a)
}

func twoprint() {
    go doprint()
    go doprint()
}
```


calling `twoprint` will call `setup` exactly once. The `setup` function will complete before either call of `print`. The result will be that "hello, world" will be printed twice.

Atomic Values

The APIs in the `sync/atomic` package are collectively "atomic operations" that can be used to synchronize the execution of different goroutines. If the effect of an atomic operation A is observed by atomic operation B, then A is synchronized before B. All the atomic operations executed in a program behave as though executed in some sequentially consistent order.

The preceding definition has the same semantics as C++'s sequentially consistent atomics and Java's volatile variables.

Finalizers

The runtime package provides a `SetFinalizer` function that adds a finalizer to be called when a particular object is no longer reachable by the program. A call to `SetFinalizer(x, f)` is synchronized before the finalization call `f(x)`.

Additional Mechanisms

The `sync` package provides additional synchronization abstractions, including condition variables, lock-free maps, allocation pools, and wait groups. The documentation for each of these specifies the guarantees it makes concerning synchronization.

Other packages that provide synchronization abstractions should document the guarantees they make too.

Incorrect synchronization

Programs with races are incorrect and can exhibit non-sequentially consistent executions. In particular, note that a read `r` may observe the value written by any write `w` that executes concurrently with `r`. Even if this occurs, it does not imply that reads happening after `r` will observe writes that happened before `w`.

In this program:

```
var a, b int
```

```
func f() {  
    a = 1  
    b = 2  
}
```

```
func g() {  
    print(b)  
    print(a)  
}
```

```
func main() {  
    go f()  
    g()  
}
```

it can happen that g prints 2 and then 0.

This fact invalidates a few common idioms.

Double-checked locking is an attempt to avoid the overhead of synchronization. For example, the twoprint program might be incorrectly written as:

```
var a string  
var done bool
```

```
func setup() {  
    a = "hello, world"  
    done = true  
}
```

```
func doprint() {  
    if !done {  
        once.Do(setup)  
    }  
    print(a)  
}
```

```
}
```

```
func twoprint() {  
    go dprint()  
    go dprint()  
}
```

but there is no guarantee that, in `dprint`, observing the write to `done` implies observing the write to `a`. This version can (incorrectly) print an empty string instead of "hello, world".

Another incorrect idiom is busy waiting for a value, as in:

```
var a string  
var done bool
```

```
func setup() {  
    a = "hello, world"  
    done = true  
}
```

```
func main() {  
    go setup()  
    for !done {  
    }  
    print(a)  
}
```

As before, there is no guarantee that, in `main`, observing the write to `done` implies observing the write to `a`, so this program could print an empty string too. Worse, there is no guarantee that the write to `done` will ever be observed by `main`, since there are no synchronization events between the two threads. The loop in `main` is not guaranteed to finish.

There are subtler variants on this theme, such as this program.

```
type T struct {  
    msg string
```

```
}

var g *T

func setup() {
    t := new(T)
    t.msg = "hello, world"
    g = t
}

func main() {
    go setup()
    for g == nil {
    }
    print(g.msg)
}
```

Even if main observes `g != nil` and exits its loop, there is no guarantee that it will observe the initialized value for `g.msg`.

In all these examples, the solution is the same: use explicit synchronization.

Incorrect compilation

The Go memory model restricts compiler optimizations as much as it does Go programs. Some compiler optimizations that would be valid in single-threaded programs are not valid in all Go programs. In particular, a compiler must not introduce writes that do not exist in the original program, it must not allow a single read to observe multiple values, and it must not allow a single write to write multiple values.

All the following examples assume that `*p` and `*q` refer to memory locations accessible to multiple goroutines.

Not introducing data races into race-free programs means not moving writes out of conditional statements in which they appear. For example, a compiler must not invert the conditional in this program:

```
*p = 1
if cond {
    *p = 2
}
```

That is, the compiler must not rewrite the program into this one:

```
*p = 2
if !cond {
    *p = 1
}
```

If `cond` is false and another goroutine is reading `*p`, then in the original program, the other goroutine can only observe any prior value of `*p` and 1. In the rewritten program, the other goroutine can observe 2, which was previously impossible.

Not introducing data races also means not assuming that loops terminate. For example, a compiler must in general not move the accesses to `*p` or `*q` ahead of the loop in this program:

```
n := 0
for e := list; e != nil; e = e.next {
    n++
}
i := *p
*q = 1
```

If `list` pointed to a cyclic list, then the original program would never access `*p` or `*q`, but the rewritten program would. (Moving `*p` ahead would be safe if the compiler can prove `*p` will not panic; moving `*q` ahead would also require the compiler proving that no other goroutine can access `*q`.)

Not introducing data races also means not assuming that called functions always return or are free of synchronization operations. For example, a compiler must not move the accesses to `*p` or `*q` ahead of the function

call in this program (at least not without direct knowledge of the precise behavior of `f`):

```
f()
i := *p
*q = 1
```

If the call never returned, then once again the original program would never access `*p` or `*q`, but the rewritten program would. And if the call contained synchronizing operations, then the original program could establish happens before edges preceding the accesses to `*p` and `*q`, but the rewritten program would not.

Not allowing a single read to observe multiple values means not reloading local variables from shared memory. For example, a compiler must not discard `i` and reload it a second time from `*p` in this program:

```
i := *p
if i < 0 || i >= len(funcs) {
    panic("invalid function index")
}
... complex code ...
// compiler must NOT reload i = *p here
funcs[i]()
```

If the complex code needs many registers, a compiler for single-threaded programs could discard `i` without saving a copy and then reload `i = *p` just before `funcs[i]()`. A Go compiler must not, because the value of `*p` may have changed. (Instead, the compiler could spill `i` to the stack.)

Not allowing a single write to write multiple values also means not using the memory where a local variable will be written as temporary storage before the write. For example, a compiler must not use `*p` as temporary storage in this program:

```
*p = i + *p/2
```

That is, it must not rewrite the program into this one:

```
*p /= 2
```

```
*p += i
```

If `i` and `*p` start equal to 2, the original code does `*p = 3`, so a racing thread can read only 2 or 3 from `*p`. The rewritten code does `*p = 1` and then `*p = 3`, allowing a racing thread to read 1 as well.

Note that all these optimizations are permitted in C/C++ compilers: a Go compiler sharing a back end with a C/C++ compiler must take care to disable optimizations that are invalid for Go.

Note that the prohibition on introducing data races does not apply if the compiler can prove that the races do not affect correct execution on the target platform. For example, on essentially all CPUs, it is valid to rewrite

```
n := 0
for i := 0; i < m; i++ {
    n += *shared
}
into:
n := 0
local := *shared
for i := 0; i < m; i++ {
    n += local
}
```

provided it can be proved that `*shared` will not fault on access, because the potential added read will not affect any existing concurrent reads or writes. On the other hand, the rewrite would not be valid in a source-to-source translator.

Conclusion

Go programmers writing data-race-free programs can rely on sequentially consistent execution of those programs, just as in essentially all other modern programming languages.

When it comes to programs with races, both

programmers and compilers should remember the advice:
don't be clever.

Why Go
Use Cases
Case Studies
Get Started
Playground
Tour
Stack Overflow
Help
Packages
Standard Library
About Go Packages
About
Download
Blog
Issue Tracker
Release Notes
Brand Guidelines
Code of Conduct
Connect
Twitter
GitHub
Slack
r/golang
Meetup
Golang Weekly
The Go Gopher
Copyright
Terms of Service
Privacy Policy
Report an Issue
System theme
Google logo
go.dev uses cookies from Google to deliver and enhance
the quality of its services and to analyze traffic. Learn
more.
Okay

The Go Memory Model document you're referring to specifies the conditions under which reads and writes to variables in different goroutines can be safely executed without causing data races. Here are the key points relevant to your question about copying a pointer:

- 1. Synchronization Requirements:** The document emphasizes that programs modifying data accessed concurrently by multiple goroutines must serialize such access to ensure that all goroutines see a consistent view of the data. This serialization can be achieved through channel operations or synchronization primitives provided by the `sync` and `sync/atomic` packages.
- 2. Atomic Operations:** For pointer copying specifically, the memory model doesn't detail this scenario explicitly. However, it mentions that atomic operations provided by the `sync/atomic` package ensure visibility of changes across goroutines. While copying a pointer is typically considered an atomic operation on most hardware (meaning the operation completes in a single step without interruption), the Go Memory Model advises using explicit synchronization to guarantee that all goroutines have a consistent view of the memory.
- 3. Data Races and Undefined Behavior:** The document defines a data race as concurrent access to the same variable by multiple goroutines, where at least one access is a write, and there's no ordering of these accesses through synchronization. Accessing a variable in such a way without proper synchronization is considered a data race, leading to undefined behavior. This means that while copying a pointer is generally safe (because the action of copying the pointer itself is atomic), accessing the pointed-to data without synchronization can cause data races if one goroutine